

# Sender Algorithm Customization

## Draft Proposal

Document #: P2999R1  
Date: 2023-11-09  
Project: Programming Language C++  
Audience: LEWG Library Evolution  
Reply-to: Eric Niebler  
<[eric.niebler@gmail.com](mailto:eric.niebler@gmail.com)>

## Contents

- [1 Introduction](#)
- [2 The Issue](#)
- [3 Proposed Design](#)
  - [3.1 Features and rationale](#)
    - [3.1.1 Dispatching via execution domain tags](#)
    - [3.1.2 Late \(sender/receiver connection-time\) customization](#)
    - [3.1.3 Early \(sender construction-time\) customization](#)
    - [3.1.4 Decomposable senders](#)
  - [3.2 Summary of proposed changes](#)
- [4 Implementation Experience](#)
- [5 Proposed Wording](#)
- [6 References](#)

## § 1 Introduction

This paper proposes some design changes to P2300 to address some shortcomings in how algorithm customizations are found.

## § 2 The Issue

The essence of the issue is this:

*Many senders do not know on what execution context they will complete, so using solely that information to find customizations (as P2300R7 does) is unsatisfactory.*

In [P2300R7], the sender algorithms (then, `let_value`, etc) are customization point objects that internally dispatch via `tag_invoke` to the correct algorithm implementation. Each algorithm has a default implementation that is used if no custom implementation is found.

Custom implementations of sender algorithms are found by asking the predecessor sender for its completion scheduler and using the scheduler as a tag for the purpose of tag dispatching. A *completion scheduler* is a scheduler that refers to the execution context on which that sender will complete.

A typical sender algorithm like then might be implemented as follows:

```
/// @brief A helper concept for testing whether an algorithm customization
/// exists
template <class AlgoTag, class SetTag, class Sender, class... Args>
concept has-customization =
    requires (Sender sndr, Args... args) {
        tag_invoke(AlgoTag(),
            get_completion_scheduler<SetTag>(get_env(sndr)),
            std::forward<Sender>(sndr),
            std::forward<Args>(args)...);
    };

/// @brief The tag type and the customization point object type for the
/// `then` sender algorithm
struct then_t {
    template <sender Sender, class Fun>
        requires /* requirements here */
    auto operator()(Sender&& sndr, Fun fun) const
    {
        // If the predecessor sender has a completion scheduler, and if we can use
        // the completion scheduler to find a custom implementation for the `then`
        // algorithm, dispatch to that. Otherwise, dispatch to the default `then`
        // implementation.
        if constexpr (has-customization<then_t, set_value_t, Sender, Fun>)
        {
            auto&& env = get_env(sndr);
            return tag_invoke(*this,
                get_completion_scheduler<set_value_t>(env),
                std::forward<Sender>(sndr),
                std::move(fun));
        }
        else
        {
            return then-sender<Sender, Fun>(std::forward<Sender>(sndr), std::move(fun));
        }
    }
};

inline constexpr then_t then {};
```

This scheme has a number of shortcomings:

1. A simple sender like `just(42)` does not know its completion scheduler. It completes on the execution context on which it is started. That is not known at the time the sender is constructed, which is when we are looking for customizations.
2. For a sender like `on( sched, then(just(), fun) )`, the nested then sender is constructed before we have specified the scheduler, but we need the scheduler to dispatch to the correct customization of then. How?
3. A composite sender like `when_all(sndr1, sndr2)` cannot know its completion scheduler in the general case. Even if `sndr1` and `sndr2` both know their completion schedulers – say, `sched1`

and sched2 respectively – the when\_all sender can complete on *either* sched1 or sched2 depending on which of sndr1 and sndr2 completes last. That is a dynamic property of the program’s execution, not suitable for finding an algorithm customization.

In cases (1) and (2), the issue is that the information necessary to find the correct algorithm implementation is not available at the time we look for customizations. In case (3), the issue is that the algorithm semantics make it impossible to know statically to what algorithm customization scheme to dispatch.

The issue described in (2) above is particularly pernicious. Consider these two programs (where `ex::` is a namespace alias for `std::execution`); the differences are highlighted:

Table 1: Algorithms customizations are found or not depending on subtle differences

| Good                                                                                                                                                                                                                                                                                                                       | Bad                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> my::thread_pool_scheduler sch = /*...*/;  // Describe some bulk work on a thread pool auto work =     ex::transfer_just(sch, data)   ex::bulk(data.size(),            [](int i, auto&amp; data) {                ++data[i];            });  // Execute the work std::this_thread::sync_wait(std::move(work)); </pre> | <pre> my::thread_pool_scheduler sch =     /*...*/;  // Describe some bulk work auto work =     ex::just(data)   ex::bulk(data.size(),            [](int i, auto&amp; data) {                ++data[i];            });  // Execute the bulk work on a thread // pool std::this_thread::sync_wait(ex::on(sch, std::move(work))); </pre> |

These two programs *should* be equivalent, but they are not. The author of the `thread_pool_scheduler` gave it a custom bulk implementation by defining:

```

namespace my {
    // customization of the bulk algorithm for the thread_pool_scheduler:
    template <ex::sender Sender, std::integral Shape, class Function>
    auto tag_invoke(ex::bulk_t,
                    thread_pool_scheduler sched,
                    Sender&& sndr,
                    Shape shape,
                    Function fun) {
        /*
         * Do bulk work in parallel
         * ...
         */
    }
}

```

This overload is found only when the bulk sender’s predecessor completes on a `thread_pool_scheduler`, which is the case for the code on the left.

In the code to the right, however, the predecessor of the bulk operation is `just(data)`, a sender that does not know where it will complete. As a result, the above customization of the bulk algorithm will

not be found, and the bulk operation will execute serially on a single thread in the thread pool. That’s almost certainly *not* what the programmer intended.

This is clearly broken and badly in need of fixing.

*Note:* On the need for async algorithms customization

It is worth asking why async algorithms need customization at all. After all, the classic STL algorithms need no customization; they dispatch using a fixed concept hierarchy to a closed set of possible implementations.

The reason is because of the open and continually evolving nature of execution contexts. There is little hope of capturing every salient attribute of every interesting execution model – CPUs, GPUs, FPGAs, etc., past, present, and future – in a fixed ontology around which we can build named concepts and immutable basis operations. Instead we do the best we can and then hedge against the future by making the algorithms customizable. For example, say we add an algorithm `std::par_algo`, but we allow that there may be an accelerator “out there” that may do `par_algo` more efficiently than the standard one, so we make `par_algo` customizable.

## § 3 Proposed Design

### § 3.1 Features and rationale

This section describes at a high level the salient features of the proposed design for sender algorithm customization, and their rationale. But in a nutshell, the basic idea is as follows:

For every invocation of a sender algorithm, the implementation looks for a customization *twice*: once immediately while the algorithm is constructing a sender to return, and once later *when the resulting sender is connect-ed with a receiver*.

It is the second look-up that is new. By looking for a customization at connect time, the dispatching logic is informed both by information from the predecessor sender(s) as well as from the receiver. It is the receiver that has information about the environment of the currently executing asynchronous operation, information that is key to picking the right customization in the cases we looked at above.

#### § 3.1.1 Dispatching via execution domain tags

As described above, the `when_all` sender doesn’t know its completion scheduler, so we cannot use the completion scheduler to find the `when_all` customization. Even if all its child senders advertise completion schedulers with the same type – say, `static_thread_pool` – `when_all` itself can’t advertise a completion scheduler because it doesn’t know that they are all the *same* `static_thread_pool`.

In the case just described, consider that we can know the completion scheduler’s *type* but not its value. So at the very least, we need to add a query about the type of the scheduler apart from its value, for times when we know one but not the other.

Once we have done that, further generalizing the query from a scheduler type to an abstract tag type is a short hop. We call this abstract tag type an *execution domain*. Several different scheduler types

may all want to use the same set of algorithm implementations; those schedulers can all use the same execution domain type for the purpose of dispatching.

If `when_all`'s child senders all share an execution domain, we know the execution domain of the `when_all` sender itself even if we don't know which scheduler it will complete on. But that no longer prevents us from dispatching to the correct implementation.

This paper proposes the addition of a forwarding `get_domain` query in the `std::execution` namespace, and that the domain is used together with the algorithm tag to dispatch to the correct algorithm implementation.

Additionally, we proposed that the `when_all` algorithm only accepts a set of senders when they all share a common domain. Likewise for `let_value` and `let_error`, we require that there is only one possible domain on which their senders may complete.

### § 3.1.2 Late (sender/receiver connection-time) customization

As described above, the sender algorithm customization points don't have all the information they need to dispatch to the correct algorithm implementation in all cases. The solution is to look again for a customization when all the information is available. That happens when the sender is connect-ed to a receiver.

This paper proposes the addition of a `transform_sender` function that is called by the connect customization point to transform a sender prior to connecting it with the receiver. The correct sender transformation is found using a property read from the receiver's environment.

The following comparison table shows how we propose to change the connect customization point (changes highlighted):

Table 2: The addition of `transform_sender` to connect

| Before                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | After                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> struct connect_t {     template &lt;receiver Receiver,               sender_in&lt;env_of_t&lt;Receiver&gt;&gt;               Sender&gt;     requires /* ... */     auto operator()(Sender&amp;&amp; sndr,                     Receiver&amp;&amp; rcvr) const {          // First, look for a         // customization of tag_invoke:         if constexpr             (tag_invocable&lt;connect_t,                           Sender, Receiver&gt;) {             return tag_invoke(*this,                                 std::forward&lt;Sender&gt;(sndr),                                 std::forward&lt;Receiver&gt;                                (rcvr));         } </pre> | <pre> struct connect_t {     template &lt;receiver Receiver,               sender_in&lt;env_of_t&lt;Receiver&gt;&gt;               Sender&gt;     requires /* ... */     auto operator()(Sender&amp;&amp; sndr,                     Receiver&amp;&amp; rcvr) const {         // Apply any sender         // transformations using the         // receiver's domain:         auto&amp;&amp; sndr2 =             transform_sender(get_env-                              domain(get_env(rcvr)),                               std::forward&lt;Sender&gt;(sndr),                               get_env(rcvr));         using Sender2 = decltype(sndr2);          // First, look for a         // customization of tag_invoke:         if constexpr             (tag_invocable&lt;connect_t,                           Sender2, Receiver&gt;) {             return tag_invoke(*this, </pre> |

| Before                                                                                                                                             | After                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> // Next, see if the sender is co_await-able: else if constexpr (is-await- connectable&lt;Sender, Receiver&gt;) {     /* ... */ } } }; </pre> | <pre> std::forward&lt;Sender2&gt;(sndr2),  std::forward&lt;Receiver&gt; (rcvr)); } // Next, see if the sender is co_await-able: else if constexpr (is-await- connectable&lt;Sender2, Receiver&gt;) {     /* ... */ } } }; </pre> |

### § 3.1.2.1 Parity with coroutines

The use of `transform_sender` in `connect` it is analagous to the use of `await_transform` in `co_await`. Glossing over some details, in a coroutine the expression `co_await expr` is “lowered” to operator `co_await(p.await_transform(expr)).await_suspend(handle-to-p)`, where `p` is a reference to coroutine’s promise. This gives the coroutine task type some say in how `co_await` expressions are evaluated.

The addition of `transform_sender` to P2300 satisfies the same need to customize the launch behavior of child async tasks. An expression like `connect(sndr, detached-receiver)` is “lowered” to `connect(transform_sender(domain, sndr, get_env(detached-receiver)), detached-receiver)`, where `domain` is a property of the receiver’s environment. This gives the receiver some say in how `connect` expressions are evaluated.

### § 3.1.2.2 Recursive transformations

The author anticipates the need to sometimes apply a transformation recursively to all of a sender’s child senders. Such a generic recursive transformation might look something like this:

```

```c++
// my_domain applies a transformation recursively
auto my_domain::transform_sender(Sender&& sndr, const Env& env) const {
    auto [tag, data, child...] = sndr;

    // Create a temporary sender with transformed children
    auto tmp = make_sender(
        tag,
        data,
        ex::transform_sender(*this, child, env)...);

    // Use the transformed children to compute a domain
    // (they all must share a domain or it's an error)
    auto&& [x, y, child2...] = tmp;
    auto domain2 = common-domain-of(child2...);

    // Use the predecessor domain to transform the temporary sender:
    return ex::transform_sender(domain2, move(tmp), env);
}
```

```

This works well until we apply this function to a sender that modifies the environment it passes to its child operations. Take the case of `on(sched, sndr)`: when it is connected to a receiver, it connects its child sender `sndr` with a receiver whose environment has been modified to show that the current scheduler is `sched` (because `on` will start `sndr` there).

But the implementation of `my_domain::transform_sender` above does not update the environment when it is recursively transforming an `on` sender's child. That means the child will be transformed with incorrect information about where it will be executing, which can change the meaning of the transformation. That's not good. Something is missing.

We need a way to ask a sender to apply its transformation to an environment. That is, in addition to `transform_sender` we need `transform_env` that can be used to fix the code above as follows (differences highlighted):

```

...c++
// my_domain applies a transformation recursively
auto my_domain::transform_sender(Sender&& sndr, const Env& env) const {
    auto [tag, data, child...] = sndr;

    // Apply any necessary transformations to the environment
    auto&& env2 = ex::transform_env(*this, sndr, env);

    // Create a temporary sender with transformed children,
    // using the transformed environment from the line above
    auto tmp = make_sender(
        tag,
        data,
        ex::transform_sender(*this, child, env2)...);

    // Use the transformed children to compute a domain
    // (they all must share a domain or it's an error)
    auto&& [x, y, child2...] = tmp;
    auto domain2 = common-domain-of(child2...);

    // Use the predecessor domain to transform the temporary sender:
    return ex::transform_sender(domain2, move(tmp), env);
}
...

```

Now expressions can generically be transformed recursively.

### § 3.1.3 Early (sender construction-time) customization

We can use `transform_sender` for early customization as well as late. The benefit of doing this is that only one set of customizations needs be written for each domain, rather than two (early and late).

This paper proposes that each algorithm constructs a default sender that implements the default behavior for that algorithm. It then passes that sender to `transform_sender` along with the sender's domain. The result of `transform_sender` is what the algorithm returns.

The following comparison table shows how we propose to change the connect customization point (changes highlighted):

Table 3: The proposed changes to the then customization point

| Before | After |
|--------|-------|
|--------|-------|

| Before                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | After                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> struct then_t {     template &lt;sender Sender, class Fun&gt;         requires /* ... */     auto operator()(Sender&amp;&amp; sndr, Fun fun)         const {         // First, use the completion scheduler to         // look for a tag_invoke         if constexpr (has-customization&lt;then_t,             set_value_t, Sender, Fun&gt;) {             auto&amp;&amp; env = get_env(sndr);             return tag_invoke(*this,                  get_completion_scheduler&lt;set_value_t&gt;                     (env),                  std::forward&lt;Sender&gt;                     (sndr),                  std::move(fun));         }         // Otherwise, use the default         // implementation:         else {             return then_sender&lt;Sender, Fun&gt;                 (std::forward&lt;Sender&gt;(sndr),                  std::move(fun));         }     } }; </pre> | <pre> struct then_t {     template &lt;sender Sender,         class Fun&gt;         requires /* ... */     auto operator()(Sender&amp;&amp;         sndr, Fun fun) const         {         // Get the domain from         // the predecessor         // sender:         auto domain = get-sender-             domain(sndr);          // Create a `then` sender         // and ask the         // predecessor to         // transform it if         // desired         return transform_sender(             domain,             then_sender&lt;Sender,                 Fun&gt;                 (std::forward&lt;Sender&gt;                     (sndr),                      std::move(fun)));         } }; </pre> |

Some algorithms are required to do some work eagerly in their default implementation (e.g., `split`, `ensure_started`). These algorithms must first create a dummy sender to pass to `transform_sender`. The “default” domain, which is used when no other domain has been specified, can transform these dummy senders and do their eager work in the process. The same mechanism is also useful to implement customizable sender algorithms whose default implementation merely lowers to a more primitive expression (e.g. `transfer(s,sch)` becomes `schedule_from(sch,s)`, and `transfer_just(sch, ts...)` becomes `just(ts...) | transfer(sch)`).

For example, here is how the `transfer_just` customization point might look after the change:

Table 4: The proposed changes to the `then` customization point

| Before                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | After                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> struct transfer_just_t {     template &lt;scheduler Scheduler, class... Ts&gt;         requires /* ... */     auto operator()(Scheduler sched, Ts&amp;&amp;...         ts) const {         // First, use the completion scheduler to         // look for a tag_invoke         if constexpr (             has-customization&lt;                 transfer_just_t, set_value_t,                 Scheduler, Ts...&gt;) {             auto&amp;&amp; env = get_env(sndr);             return tag_invoke(*this, </pre> | <pre> struct transfer_just_t {     template &lt;scheduler Scheduler,         class... Ts&gt;         requires /* ... */     auto operator()(Scheduler sched,         Ts&amp;&amp;... ts) const {         // Get the predecessor's domain         auto domain = get-sender-             domain(sndr);          // Construct a transfer_just sender         // and transform it         return transform_sender(             domain, </pre> |



| Before                                                                                                                                                                                                                                                                                  | After                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> get_completion_scheduler&lt;set_value_t&gt; (env),          std::move(sched),         std::forward&lt;Ts&gt; (ts)...); } // Otherwise, use the default // implementation: else {     return just(std::forward&lt;Ts&gt;(ts)...           transfer(std::move(sched))); } }; </pre> | <pre> make_sender(*this,     tuple{std::move(sched),         std::forward&lt;Ts&gt;(ts)...}); }  // default_domain::transform_sender // dispatches here: template &lt;class Sender, class Env&gt; auto transform_sender(Sender&amp;&amp; sndr,     const Env&amp; env) const {     auto [tag, data] =         std::forward&lt;Sender&gt;(sndr);     auto [sched, ts...] =         std::move(data);     return just(std::move(ts)...           transfer(std::move(sched))); } }; </pre> |

Some algorithms are entirely eager with no lazy component, like `sync_wait` and `start_detached`. For these, “transforming” a sender isn’t what you want; you want to dispatch to an eager algorithm that will actually *consume* the sender. We can use domains to dispatch to the correct implementation for those as well. This paper proposes the addition of an `apply_sender` function.

The following table describes the differences between `transform_sender` and `apply_sender`:

Table 5: The differences between `transform_sender` and `apply_sender`

| <code>transform_sender</code>                                                                        | <code>apply_sender</code>                                                                         |
|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| — always called with either a sender or a sender+env                                                 | — called with a sender and an arbitrary set of additional arguments                               |
| — always returns a sender                                                                            | — can return anything                                                                             |
| — has a sensible default implementation: identity                                                    | — has no sensible default implementation                                                          |
| — useful for lazy or partly lazy algorithms (e.g., <code>then</code> , <code>ensure_started</code> ) | — useful for fully eager algorithms (e.g., <code>start_detached</code> , <code>sync_wait</code> ) |

To permit third parties to author customizable sender algorithms with partly or fully eager behavior, the mechanism by which the default domain finds the default `transform_sender` and `apply_sender` implementations shall be specified: they both dispatch to similarly named functions on the tag type of the input sender; i.e., `default_domain().transform_sender(sndr, env)` is equal to `tag_of_t<decltype(sndr)>().transform_sender(sndr, env)`.

### § 3.1.4 Decomposable senders

For the `transform_sender` customization point to be useful, we need a way to access the constituent pieces of a sender and re-assemble it from (possibly transformed) pieces. Senders, like coroutines, generally begin in a “suspended” state; they merely curry their algorithm’s arguments into a subsequent call to connect. These “suspended” senders are colloquially known as *lazy* senders.

Each lazy sender has an associated algorithm tag, a (possibly empty) set of auxiliary data and a (possibly empty) set of child senders; *e.g.*, the sender returned from `then(snd, fun)` has `then_t` as its tag, the set `[fun]` as its auxiliary data, and `[snd]` as its set of child senders, while `just(42, 3.14)` has `just_t` as its tag, `[42, 3.14]` as its data set and `[]` as its child set.

This paper proposes to use structured bindings as the API for decomposing a lazy sender into its tag, data, and child senders:

```
auto&& [tag, data, ...children] = sndr;
```

[P1061R5], currently in Core wording review for C++26, permits the declaration of variadic structured bindings like above, making this syntax very appealing.

Not all senders are required to be decomposable, although all the “standard” lazy senders shall be. There needs to be a syntactic way to distinguish between decomposable and ordinary, non-decomposable senders (decomposable senders subsuming the sender concept).

There is currently no trait for determining whether a type can be the initializer of a structured binding. However, EWG has already approved [P2141R1] for C++26, and with it such a trait could be built, giving us a simple way to distinguish between decomposable and non-decomposable senders.

If P2141 is not adopted for C++26, we will need some other syntactic way to opt-in. One possibility is to require that the sender type’s nested `is_sender` type shall have some known, standard tag type as a base class to signify that that sender type can be decomposed.

#### § 3.1.4.1 Recomposing senders

After decomposing a sender, it is often desirable to re-compose it from its modified constituents. No separate API for reconstituting senders is necessary though. It is enough to construct a decomposable sender of some arbitrary type and then pass it to `transform_sender` with an execution domain to place it in its final form.

Consider the case where `my_domain::transform_sender()` is passed `your_sender<Children...>`. It unpacks it into its tag/data/children constituents and munges the children somehow. It then wants to reconstruct a `your_sender` from the munged children. Instead, it constructs *arbitrary-sender*{tag, data, munged\_children} and passes it to `execution::transform_sender()` along with `your_domain`, the domain associated with `your_sender`. That presumably will transform the *arbitrary-sender* back into a `your_sender`.

## § 3.2 Summary of proposed changes

In condensed form, here are the changes this paper is proposing:

1. Add a `default_domain` type for use when no other domain is determinable.
2. Add a new `get_domain(env) -> domain-tag` forwarding query.
3. Add a new `transform_sender(domain, sender [, env]) -> sender` API. This function is not itself customizable, but it will be used for both early customization (at sender construction-time) and late customization (at sender/receiver connection-time).

*Early customization:*

- called from within each sender algorithm’s customization point object
- replaces the current mechanism of tag-dispatching to a sender factory function using the completion scheduler as a tag
- called without an environment argument
- *domain* is derived from the sender by trying the following in order:

1. `get_domain(get_env(sender))`
2. `get_domain(get_completion_scheduler<completion-tag>(get_env(sender)))`, where *completion-tag* is one of `set_value_t`, `set_error_t`, or `set_stopped_t` depending on the algorithm
3. `default_domain()`

*Late customization:*

- called from the connect customization point object before tag-dispatching with `connect_t` to `tag_invoke`
- called with the receiver’s environment
- *domain* is derived from the receiver by trying the following in order:

1. `get_domain(get_env(receiver))`
2. `get_domain(get_scheduler(get_env(receiver)))`
3. `default_domain()`

`transform_sender(domain, sender [, env])` returns the first of these that is well-formed:

- `domain.transform_sender(sender [, env])`
- `default_domain().transform_sender(sender [, env])`
- *sender*

4. Add a `transform_env(domain, sender, env) -> env'` API in support of generic recursive sender transformations. The *domain* argument is determined from *env* as for `transform_sender`

`transform_env(domain, sender, env)` returns the first of these that is well-formed:

- `domain.transform_env(sender, env)`
- `default_domain().transform_env(sender, env)`

5. The standard, “lazy” sender types (i.e., those returned from sender factory and adaptor functions) return sender types that are decomposable using structured bindings into its `[tag, data, ...children]` components.
6. A call to the `when_all` algorithm should be ill-formed unless all of the sender arguments have the same domain type (as determined for senders above). The resulting `when_all` sender should publish that domain via the sender’s environment.
7. The `on(sch, sndr)` algorithm should be specified in terms of transfer so as to pick up any late customization of the transfer algorithm. (This amounts to changing `schedule(sch)` to

`transfer_just(sch)` in `[exec.on]/3.2.2`). Additionally, it should replace the domain in the receiver's environment with the domain of `sch`.

8. The sender factories `just`, `just_error`, and `just_stopped` need their tag types to be specified. Name them `just_t`, `just_error_t`, and `just_stopped_t`.
9. In the algorithm `let_value(sndr, fun)`, if the predecessor sender `sndr` has a completion scheduler for `set_value`, then the receiver connected to the secondary sender (the one returned from `fun` when called with `sndr`'s results) shall expose that scheduler as the current scheduler of the receiver's environment.

In other words, if the predecessor sender `sndr` completes with values `vs...`, then the result of `fun(vs...)` will be connected to a receiver `r` such that `get_scheduler(get_env(r))` is equal to `get_completion_scheduler<set_value_t>(get_env(sndr))`.

The same is true also of the domain query: `get_domain(get_env(r))` is equal to the domain of the predecessor sender as computed by the steps in (2) above.

So for `let_value`, likewise also for `let_error` and `let_stopped`, using `set_error_t` and `set_stopped` respectively when querying for the predecessor sender's completion scheduler and domain.

10. The `schedule_from(sched, sndr)` algorithm should return a sender `s` such that `get_domain(get_env(s))` is equal to `get_domain(sched)`.
11. The following customizable algorithms, whose default implementations must do work before returning the result sender, will have their work performed in overloads of `default_domain::transform_sender`:

- `split`
- `ensure_started`

12. The following customizable algorithms, whose default implementations are trivially expressed in terms of other more primitive operations, will be lowered into their primitive forms by overloads of `default_domain::transform_sender`:

- `transfer`
- `transfer_just`
- `transfer_when_all`
- `transfer_when_all_with_variant`
- `when_all_with_variant`

13. In the algorithm `let_value(snd, fun)`, all of the sender types that the input function `fun` might return – the set of potential result senders – must all have the same domain; otherwise, the call to `let_value` is ill-formed.

Ideally, the `let_value` sender would report the result senders' domain as its domain, however we don't know the set of completions until the `let_value` sender is connected to a receiver; hence, we also don't know the set of potential result senders or their domains. Instead, we require that all the result senders share an execution domain with the predecessor sender. If they differ, connect is ill-formed.

For example, consider the following sender:

```
auto snd =
  read(get_scheduler)
  | transfer(schA)
  | let_value([],(auto schB){ return schedule(schB); })
```

This reads the current scheduler from the receiver, transfers execution to schA, and then (indirectly, through `let_value`) transitions onto the scheduler read from the receiver (schB). This sender can be connect-ed only to receivers R for which the scheduler `get_scheduler(get_env(R))` has the same execution domain as that of schA.

Likewise for `let_error` and `let_stopped`.

This solution is not ideal. I am currently working on a more flexible solution, but I'm not yet sufficiently confident in it to propose it here.

14. Add a new `apply_sender(domain, tag, sender, args...) -> result` API. Like `transform_sender`, this function is not itself customizable, but it will be used to customize sender consuming algorithms such as `start_detached` and `sync_wait`.

- called from within each consuming sender algorithm's customization point object
- replaces the current mechanism of tag-dispatching to a custom implementation using the completion scheduler as a tag
- called the sender's execution domain, the algorithm's tag, the sender, and any additional algorithm arguments
- `domain` is determined as for `transform_sender`

`apply_sender(domain, tag, sender, args...)` returns the first of these that is well-formed:

- `domain.apply_sender(tag, sender, args...)`
- `default_domain().apply_sender(tag, sender, args...)`

15. The following customizable sender-consuming algorithms will have their default implementations in overloads of `default_domain::apply_sender`:

- `start_detached`
- `sync_wait`

## § 4 Implementation Experience

*Has it been implented?* YES. The design changes herein proposed are implemented in the main branch of [\[stdexecgithub\]](#), the reference implementation. The bulk of the changes including `get_domain`, `transform_sender`, and the changes to connect have been shipping since [this commit](#) on August 3, 2023 which changed the `static_thread_pool` scheduler to use `transform_sender` to parallelize the bulk algorithm.

## § 5 Proposed Wording

The following proposed changes are relative to [\[P2300R7\]](#).

[ Editor's note: Change §11.4 [exec.syn] as follows: ]

```
// [exec.queries], queries
enum class forward_progress_guarantee;
namespace queries { // exposition only
    struct get_domain_t;
    struct get_scheduler_t;
    struct get_delegatee_scheduler_t;
    struct get_forward_progress_guarantee_t;
    template<class CPO>
        struct get_completion_scheduler_t;
}

using queries::get_domain_t;
using queries::get_scheduler_t;
using queries::get_delegatee_scheduler_t;
using queries::get_forward_progress_guarantee_t;
using queries::get_completion_scheduler_t;
inline constexpr get_domain_t get_domain{};
inline constexpr get_scheduler_t get_scheduler{};
inline constexpr get_delegatee_scheduler_t get_delegatee_scheduler{};
        inline                constexpr                get_forward_progress_guarantee_t
            get_forward_progress_guarantee{};
template<class CPO>
    inline constexpr get_completion_scheduler_t<CPO> get_completion_scheduler{};

// [exec.domain.default], domains
struct default_domain;
```

[ Editor's note: ... and ... ]

```
template<class S, class E = empty_env>
    requires sender_in<S, E>
    inline constexpr bool sends_stopped = see below;

template <sender Sender>
using tag_of t = see below;

// [exec.snd.transform], sender transformations
template <class Domain, sender Sender, class... Env>
requires (sizeof...(Env) <= 1).
constexpr sender decltype(auto) transform sender(Domain dom, Sender&& sndr,
const Env&... env) noexcept(see below);

template <class Domain, sender Sender, class Env>
constexpr decltype(auto) transform env(Domain dom, Sender&& sndr, Env&& env)
noexcept;

// [exec.snd.apply], sender algorithm application
template <class Domain, class Tag, sender Sender, class... Args>
constexpr decltype(auto) apply_sender(Domain dom, Tag, Sender&& sndr, Args&&...
args) noexcept(see below);

// [exec.connect], the connect sender algorithm
namespace senders-connect { // exposition only
    struct connect_t;
}
using senders-connect::connect_t;
inline constexpr connect_t connect{};
```

[ Editor's note: ... and ... ]

```
// [exec.factories], sender factories
namespace senders-factories { // exposition only
    struct just_t;
    struct just_error_t;
    struct just_stopped_t;
    struct schedule_t;
    struct transfer_just_t;
}
using sender-factories::just_t;
using sender-factories::just_error_t;
using sender-factories::just_stopped_t;
inline constexpr unspecifiedjust_t just{};
inline constexpr unspecifiedjust_error_t just_error{};
inline constexpr unspecifiedjust_stopped_t just_stopped{};
```

[ Editor's note: After §11.5.4 [exec.get.stop.token], add the following new subsection: ]

### §11.5.? execution::get\_domain [exec.get.domain]

1. get\_domain asks an object for an associated execution domain tag.
2. The name get\_domain denotes a query object. For some subexpression r, get\_domain(r) is expression-equivalent to *mandate-nothrow-call*(tag\_invoke, get\_domain, as\_const(r)), if this expression is well-formed.
3. std::forwarding\_query(execution::get\_domain) is true.
4. get\_domain() (with no arguments) is expression-equivalent to execution::read(get\_domain) ([exec.read]).

[ Editor's note: To section §11.6 [exec.sched], insert a new paragraph between 6 and 7 as follows: ]

7. For a given scheduler expression s, if the expression get\_domain(s) is well-formed, then the expression get\_domain(get\_env(schedule(s))) is also well-formed and has the same type.

[ Editor's note: To section §11.9.1 [exec.snd.concepts], after paragraph 4, add two new paragraphs as follows: ]

5. Let s be an expression such that decltype((s)) is S. The type tag\_of\_t<S> is as follows:
  - If the declaration auto&& [tag, data, ...children] = s; would be well-formed, tag\_of\_t<S> is an alias for decltype(auto(tag)).
  - Otherwise, tag\_of\_t<S> is ill-formed.

[ Editor's note: There is no way in standard C++ to determine whether the above declaration is well-formed without causing a hard error, so this presumes compiler magic. However, the author anticipates the adoption of [P2141R1], which makes it possible to implement this purely in the library. P2141 has already been approved by EWG for C++26. ]

6. Let sender-for be an exposition-only concept defined as follows:

```
template <class Sender, class Tag>
concept sender-for =
    sender<Sender> &&
    same_as<tag_of_t<Sender>, Tag>;
```

[ Editor's note: After §11.9.2 [exec.awaitables], add the following new subsections: ]

### §11.9.? execution::default\_domain [exec.domain.default]

```
struct default_domain {
    template <sender Sender, class... Env>
        requires (sizeof...(Env) <= 1)
        static constexpr sender decltype(auto) transform_sender(Sender&& sndr, const
            Env&... env) noexcept(see below);

    template <sender Sender, class Env>
        static constexpr decltype(auto) transform_env(Sender&& sndr, Env&& env)
            noexcept;

    template <class Tag, sender Sender, class... Args>
        static constexpr decltype(auto) apply_sender(Tag, Sender&& sndr, Args&&...
            args) noexcept(see below);
};
```

#### §11.9.?1 Static members [exec.domain.default.statics]

```
template <sender Sender, class... Env>
    requires (sizeof...(Env) <= 1)
    constexpr sender decltype(auto) default_domain::transform_sender(Sender&& sndr,
        const Env&... env) noexcept(see below);
```

*Returns:* tag\_of\_t<Sender>().transform\_sender(std::forward<Sender>(sndr), env...) if that expression is well-formed; otherwise, std::forward<Sender>(sndr).

*Remarks:* The exception specification is equivalent to:

```
noexcept(tag_of_t<Sender>().transform_sender(std::forward<Sender>(sndr),
    env...))
```

if that expression is well-formed; otherwise, true;

```
template <sender Sender, class Env>
    constexpr decltype(auto) default_domain::transform_env(Sender&& sndr, Env&& env)
        noexcept;
```

*Returns:* tag\_of\_t<Sender>().transform\_env(std::forward<Sender>(sndr), std::forward<Env>(env)) if that expression is well-formed; otherwise, static\_cast<Env>(std::forward<Env>(env)).

*Mandates:* The selected expression in *Returns:* is not potentially throwing.

```
template <class Tag, sender Sender, class... Args>
    static constexpr decltype(auto) default_domain::apply_sender(Tag, Sender&& sndr,
        Args&&... args) noexcept(see below);
```

*Returns:* Tag().apply\_sender(std::forward<Sender>(sndr), std::forward<Args>(args)...) if that expression is well-formed; otherwise, this function shall not participate in overload resolution.

*Remarks:* The exception specification is equivalent to:

```
noexcept(Tag().apply_sender(std::forward<Sender>(sndr),
    std::forward<Args>(args)...))
```

### §11.9.? execution::transform\_sender [exec.snd.transform]



```
template <class Domain, sender Sender, class... Env>
    requires (sizeof...(Env) <= 1)
    constexpr sender decltype(auto) transform_sender(Domain dom, Sender&& sndr, const
        Env&... env) noexcept(see below);
```

*Returns:* `dom.transform_sender(std::forward<Sender>(sndr), env...)` if that expression is well-formed; otherwise, `default_domain().transform_sender(std::forward<Sender>(sndr), env...)`.

*Remarks:* The exception specification is equivalent to:

```
noexcept(dom.transform_sender(std::forward<Sender>(sndr), env...))
```

if that expression is well-formed; otherwise,

```
noexcept(default_domain().transform_sender(std::forward<Sender>(sndr),
    env...))
```

```
template <class Domain, sender Sender, class Env>
    constexpr decltype(auto) transform_env(Domain dom, Sender&& sndr, Env&& env)
    noexcept;
```

*Returns:* `dom.transform_sender(std::forward<Sender>(sndr), std::forward<Env>(env))` if that expression is well-formed; otherwise, `default_domain().transform_sender(std::forward<Sender>(sndr), std::forward<Env>(env))`.

### §11.9.? `execution::apply_sender` [`exec.snd.apply`]

```
template <class Domain, class Tag, sender Sender, class... Args>
    constexpr decltype(auto) apply_sender(Domain dom, Tag, Sender&& sndr, Args&&...
        args) noexcept(see below);
```

*Returns:* `dom.apply_sender(Tag(), std::forward<Sender>(sndr), std::forward<Args>(args)...) if that expression is well-formed; otherwise, default_domain().apply_sender(Tag(), std::forward<Sender>(sndr), std::forward<Args>(args)...) if that expression is well-formed; otherwise, this function shall not participate in overload resolution.`

*Remarks:* The exception specification is equivalent to:

```
noexcept(dom.apply_sender(Tag(), std::forward<Sender>(sndr),
    std::forward<Args>(args)...))
```

if that expression is well-formed; otherwise,

```
noexcept(default_domain().apply_sender(Tag(), std::forward<Sender>(sndr),
    std::forward<Args>(args)...))
```

[ Editor's note: Add a paragraph to §11.9 [`exec.snd`] ]

1. This section makes use of the following exposition-only entities.

```
1. template <class Tag, class Env, class Default>
    constexpr decltype(auto) query-with-default(Tag, const Env& env,
        Default&& value) noexcept(see below);
```

*Effects:* Equivalent to:

— return `Tag()(env)`; if that expression is well-formed,

— return `static_cast<Default>(std::forward<Default>(value))`; otherwise.

*Remarks:* The expression in the `noexcept` clause is:

```
is_invocable_v<Tag, const Env&> ? is_nothrow_invocable_v<Tag,
const Env&>
:
is_nothrow_constructible_v<Default, Default>
```

```
2. template <class Env, class Tag = get_scheduler_t>
constexpr auto get-env-domain(const Env& env, Tag tag = {}) noexcept;
```

*Effects:* Equivalent to:

— return `get_domain(env)`; if that expression is well-formed,

— otherwise, return `get_domain(tag(env))`; if that expression is well-formed,

— otherwise, return `default_domain()`;

```
3. template <class Sender, class Tag = set_value_t>
constexpr auto get-sender-domain(const Sender& sndr, Tag = {}) noexcept;
```

*Effects:* Equivalent to: return `get-env-domain(get_env(sndr), get_completion_scheduler<Tag>())`;

```
4. template <class... T>
struct tuple-like {
    T0 t0; // exposition only
    T1 t1; // exposition only
    ...
    Tn-1 tn-1; // exposition only
};
```

— [ *Note:* An expression of type *tuple-like* is usable as the initializer of a structured binding declaration [dcl.struct.bind]. — *end note* ]

```
5. template <semiregular Tag, movable-value Data, sender... Child>
constexpr auto make-sender(Tag, Data&& data, Child&&... child);
```

*Returns:* A prvalue of type `basic-sender<Tag, decay_t<Data>, decay_t<Child>...>` where the *tag* member has been default-initialized and the *data* and *child<sub>n</sub>...* members have been direct initialized from their respective forwarded arguments, where *basic-sender* is the following exposition-only class template except as noted below:

```
template <class Tag, class Data, class... Child> // arguments
are not associated entities ([lib.templ-heads])
struct basic-sender : unspecified {
    using is_sender = unspecified;

    [[no_unique_address]] Tag tag; // exposition only
    Data data; // exposition only
    Child0 child0; // exposition only
    Child1 child1; // exposition only
    ...
    Childn-1 childn-1; // exposition only
};
```

- It is unspecified whether instances of *basic-sender* can be aggregate initialized.
- The unspecified base type has no non-static data members. It may define member functions or hidden friend functions ([hidden.friends]).
- [ *Note:* An expression of type *basic-sender* is usable as the initializer of a structured binding declaration [dcl.struct.bind]. — *end note* ]

```
6. template <class Domain, class... Env>
    auto make-transformer-fn(Domain, const Env&... env) {
        return [&<class Tag>(Tag) {
            return [&<class... Args>(Args&&... args) -> decltype(auto) {
                return transform_sender(Domain(), Tag()(std::forward<Args>
                    (args)...), env...);
            };
        }];
    }
}
```

[ Editor's note: To §11.9.5.2 [exec.just], add a paragraph 6 as follows: ]

6. When used as the initializer of a structured binding declaration, expressions of type *just-sender*<Tag, Ts...> behave as do expressions of type *basic-sender*<Tag, tuple-like<Ts...>>.

[ Editor's note: Change §11.9.5.3 [exec.transfer.just] as follows (some identifiers in this section have had their names changed for the sake of clarity; the name changes have not been marked up): ]

2. The name `transfer_just` denotes a customization point object. For some subexpression `sch` and pack of subexpressions `vs`, let `Sch` be `decltype((sch))` and let `Vs` be the template parameter pack `decltype((vs))...`. If `Sch` does not satisfy *scheduler*, or any type `V` in `Vs` does not satisfy *movable-value*, `transfer_just(sch, vs...)` is ill-formed. Otherwise, `transfer_just(sch, vs...)` is expression-equivalent to:

```
transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(transfer_just, tuple-like{sch, vs...}));
```

1. ~~`tag_invoke(transfer_just, sch, vs...)`, if that expression is valid.~~
3. Let `as` be a pack of rvalue subexpressions of types `decay_t<Vs>...` referring to objects direct-initialized from `vs`. If ~~the function selected by `tag_invoke` a sender `S` returned from `transfer_just(sch, vs...)` is connected with a receiver `R` with environment `E` such that `transform_sender(get-env-domain(E), S, E)`~~ does not return a sender whose asynchronous operations execute value completion operations on an execution agent belonging to the execution resource associated with `sch`, with value result datums `as`, the behavior of calling ~~`transfer_just(sch, vs...)`~~ `connect(S, R)` is undefined.

— *Mandates:* `sender_of<RS, set_value_t(decay_t<Vs>...), E>`, where ~~`RS`~~ is the type of ~~the `tag_invoke` expression above~~ `transfer_just(sch, vs...)`, and `E` is the type of an environment.

2. Otherwise, `transfer(just(vs...), sch)`.
4. For some subexpression `sch` and pack of subexpressions `vs`, let `s` be a subexpression referring to an object returned from `transform_just(sch, vs...)` or a copy of such. Then

`get_completion_scheduler<set_value_t>(get_env(s)) == sch is true,`  
`get_completion_scheduler<set_stopped_t>(get_env(s)) == sch is true, and`  
`get_domain(get_env(s))` is expression-equivalent to `get_domain(sch)`.

- Let  $s$  and  $e$  be subexpressions such that  $S$  is `decltype((s))`. If *sender-for* $\langle S, \text{transfer\_just\_t} \rangle$  is false, then the expression `transfer_just_t().transform_sender(s, e)` is ill-formed; otherwise, it is equal to:

```

const auto& env = e;
auto domain = get-env-domain(env);
auto [tag, data] = s;
auto& [sched, ...vs] = data;
auto tfx = make-transformer-fn(domain, env);
return tfx(transfer)(tfx(just)(std::move(vs)...), std::move(sched));

```

[ Note: This causes the `transform_just(sch, vs...)` sender to become `transform(just(vs...), sch)` when it is connected with a receiver whose execution domain does not customize `transform_just`. — end note ]

[ Editor's note: Update §11.9.6.3 [exec.on] as follows: ]

- on adapts an input sender into a sender that will start on an execution agent belonging to a particular scheduler's associated execution resource.
- Let *replace-scheduler*( $e, sch$ ) be an expression denoting an object  $e'$  such that `get_scheduler(e')` returns a copy of `sch`, `get_domain(e')` is expression-equivalent to `get_domain(sch)`, and `tag_invoke(tag, e', args...)` is expression-equivalent to `tag(e, args...)` for all arguments `args...` and for all `tag` whose type satisfies *forwarding-query* and is not `get_scheduler_t` or `get_domain_t`.
- The name `on` denotes a customization point object. For some subexpressions `sch` and  $s$ , let  $Sch$  be `decltype((sch))` and  $S$  be `decltype((s))`. If  $Sch$  does not satisfy *scheduler*, or  $S$  does not satisfy *sender*, `on(sch, s)` is ill-formed. Otherwise, the expression `on(sch, s)` is expression-equivalent to:

```

transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(on, sch, s));

```

- ~~`tag_invoke(on, sch, s)`, if that expression is valid. If the function selected above~~ If a sender  $S$  returned from `on(sch, s)` is connected with a receiver  $R$  with environment  $E$  such that `transform_sender(get-env-domain(E), S, E)` does not return a sender which starts  $s$  on an execution agent of the associated execution resource of `sch` when started, the behavior of calling ~~`on(sch, s)`~~ `connect(S, R)` is undefined.  
  
— ~~Mandates: The type of the `tag_invoke` expression above satisfies *sender*.~~
- Let  $s$  and  $e$  be subexpressions such that  $S$  is `decltype((s))`. If *sender-for* $\langle S, \text{on } t \rangle$  is false, then the expression `on t().transform_sender(s, e)` is ill-formed; otherwise, it returns ~~Otherwise, constructs~~ a sender  $s_1$ . ~~When~~ such that when  $s_1$  is connected with some receiver `out_r`, it:

- Constructs a receiver  $r$  such that:

1. When `set_value(r)` is called, it calls `connect(s, r2)`, where `r2` is as specified below, which results in `op_state3`. It calls `start(op_state3)`. If any of these throws an exception, it calls `set_error` on `out_r`, passing `current_exception()` as the second argument.
2. `set_error(r, e)` is expression-equivalent to `set_error(out_r, e)`.
3. `set_stopped(r)` is expression-equivalent to `set_stopped(out_r)`.
4. `get_env(r)` is expression-equivalent to `get_env(out_r)`.
2. Calls `schedule(sch)transfer_just(sch)`, which results in `s2`. It then calls `connect(s2, r)`, resulting in `op_state2`.
3. `op_state2` is wrapped by a new operation state, `op_state1`, that is returned to the caller.
4. `r2` is a receiver that wraps a reference to `out_r` and forwards all completion operations to it. In addition, `get_env(r2)` returns `replace-scheduler(e, sch)`.
5. When `start` is called on `op_state1`, it calls `start` on `op_state2`.
6. The lifetime of `op_state2`, once constructed, lasts until either `op_state3` is constructed or `op_state1` is destroyed, whichever comes first. The lifetime of `op_state3`, once constructed, lasts until `op_state1` is destroyed.
5. Let `s` and `e` be subexpressions such that `S` is `decltype((s))`. If `sender-for<S, on t>` is false, then the expression on `t().transform env(s, e)` is ill-formed; otherwise, let `sch` be the scheduler used to construct `s`. on `t().transform env(s, e)` is equal to `replace-scheduler(e, sch)`.
6. Given subexpressions `s1` and `e`, where `s1` is a sender returned from `on` or a copy of such, let `S1` be `decltype((s1))`. Let `E'` be `decltype((replace-scheduler(e, sch)))`. Then the type of `tag_invoke(get_completion_signatures, s1, e)` shall be:

```
make_completion_signatures<
  copy_cvref_t<S1, S>,
  E',
  make_completion_signatures<
    schedule_result_t<Sch>,
    E,
    completion_signatures<set_error_t(exception_ptr)>,
    no-value-completions>>;
```

where `no-value-completions<As...>` names the type `completion_signatures<>` for any set of types `As...`

[ Editor's note: Update §11.9.6.4 [exec.transfer] as follows: ]

1. `transfer` adapts a sender into a sender with a different associated `set_value` completion scheduler. [ Note: It results in a transition between different execution resources when executed. — end note ]
2. The name `transfer` denotes a customization point object. For some subexpressions `sch` and `s`, let `Sch` be `decltype((sch))` and `S` be `decltype((s))`. If `Sch` does not satisfy `scheduler`, or `S`

does not satisfy sender, `transfer(s, sch)` is ill-formed. Otherwise, the expression `transfer(s, sch)` is expression-equivalent to:

```
transform_sender(
  get-sender-domain(s),
  make-sender(transfer, sch, s));
```

1. `tag_invoke(transfer, get_completion_scheduler<set_value_t>(get_env(s)), s, sch)`, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies sender.

2. Otherwise, `tag_invoke(transfer, s, sch)`, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies sender.

3. Otherwise, `schedule_from(sch, s)`.

~~If the function selected above~~ If a sender `S` returned from `transfer(s, sch)` is connected with a receiver `R` with environment `E` such that `transform_sender(get-env-domain(E), S, E)` does not return a sender ~~which~~ that is a result of a call to `transform_sender(get-env-domain(E), schedule_from(sch, s2), E)`, where `s2` is a sender ~~which~~ that sends values ~~equivalent~~ equal to those sent by `s`, the behavior of calling ~~`transfer(s, sch)`~~ `connect(S, R)` is undefined.

3. For a sender `t` returned from `transfer(s, sch)`, `get_env(t)` shall return a queryable object `q` such that `get_domain(q)` is expression-equivalent to `get_domain(sch)` and `get_completion_scheduler<CPO>(q)` returns a copy of `sch`, where `CPO` is either `set_value_t` or `set_stopped_t`. The `get_completion_scheduler<set_error_t>` query is not implemented, as the scheduler cannot be guaranteed in case an error is thrown while trying to schedule work on the given scheduler object. For all other query objects `Q` whose type satisfies *forwarding-query*, the expression `Q(q, args...)` shall be equivalent to `Q(get_env(s), args...)`.

4. Let `s` and `e` be subexpressions such that `S` is `decltype(s)`. If `sender-for<S, transfer_t>` is false, then the expression `transfer_t().transform_sender(s, e)` is ill-formed; otherwise, it is equal to:

```
const auto& env = e;
auto domain = get-env-domain(env);
auto [tag, data, child] = s;
auto tfx = make-transformer-fn(domain, env);
return tfx(schedule_from)(std::move(data), std::move(child));
```

[ *Note:* This causes the `transfer(s, sch)` sender to become `schedule_from(sch, s)` when it is connected with a receiver whose execution domain does not customize transfer. — *end note* ]

[ Editor's note: Update §11.9.6.5 [exec.schedule.from] as follows: ]

1. `schedule_from` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource. [ *Note:* `schedule_from` is not meant to be used in user code; it is used in the implementation of `transfer`. — *end note* ]

2. The name `schedule_from` denotes a customization point object. For some subexpressions `sch` and `s`, let `Sch` be `decltype((sch))` and `S` be `decltype((s))`. If `Sch` does not satisfy `scheduler`, or `S` does not satisfy `sender`, `schedule_from(sch, s)` is ill-formed. Otherwise, the expression `schedule_from(sch, s)` is expression-equivalent to:

```
transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make-schedule-from-sender(sch, s));
```

where `make-schedule-from-sender(sch, s)` is expression-equivalent to `make-sender(schedule_from, sch, s)` and returns a sender object `s2` that behaves as follows:

1. `tag_invoke(schedule_from, sch, s)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that completes on an execution agent belonging to the associated execution resource of `sch` and completing with the same async result ([`async.ops`]) as `s`, the behavior of calling `schedule_from(sch, s)` is undefined.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

1. ~~Otherwise, constructs a sender `s2`:~~ When `s2` is connected with some receiver `out_r`, it:

1. Constructs a receiver `r` such that when a receiver completion operation `Tag(r, args...)` is called, it decay-copies `args...` into `op_state` (see below) as ~~`args`~~`args2`... and constructs a receiver `r2` such that:

1. When `set_value(r2)` is called, it calls `Tag(out_r, std::move(argsargs2)...) .`

2. `set_error(r2, e)` is expression-equivalent to `set_error(out_r, e)`.

3. `set_stopped(r2)` is expression-equivalent to `set_stopped(out_r)`.

4. `get_env(r2)` is equal to `get_env(r)`.

It then calls `schedule(sch)`, resulting in a sender `s3`. It then calls `connect(s3, r2)`, resulting in an operation state `op_state3`. It then calls `start(op_state3)`. If any of these throws an exception, it catches it and calls `set_error(out_r, current_exception())`. If any of these expressions would be ill-formed, then `Tag(r, args...)` is ill-formed.

2. Calls `connect(s, r)` resulting in an operation state `op_state2`. If this expression would be ill-formed, `connect(s2, out_r)` is ill-formed.

3. Returns an operation state `op_state` that contains `op_state2`. When `start(op_state)` is called, calls `start(op_state2)`. The lifetime of `op_state3` ends when `op_state` is destroyed.

2. [ Editor's note: This para is taken from the removed para (1) above. ] ~~If the function selected by `tag_invoke`~~If a sender `S` returned from `schedule_from(sch, s)` is connected with a receiver `R` with environment `E` such that `transform_sender(get_env-domain(E), S, E)` does not return a sender that completes on an execution agent belonging to the associated execution resource of `sch` and ~~completing~~ with the same

async result ([*async.ops*]) as *s*, the behavior of calling ~~*schedule\_from(sch, s)*~~ *connect(S, R)* is undefined.

- Given subexpressions *s2* and *e*, where *s2* is a sender returned from *schedule\_from* or a copy of such, let *S2* be *decltype((s2))* and let *E* be *decltype((e))*. Then the type of *tag\_invoke(get\_completion\_signatures, s2, e)* shall be:

```
make_completion_signatures<
    copy_cvref_t<S2, S>,
    E,
    make_completion_signatures<
        schedule_result_t<Sch>,
        E,
        potentially-throwing-completions,
        no-completions>,
        value-completions,
        error-completions>;
```

where *potentially-throwing-completions*, *no-completions*, *value-completions*, and *error-completions* are defined as follows:

```
template <class... Ts>
using all-nothrow-decay-copyable =
    boolean_constant<(is_nothrow_constructible_v<decay_t<Ts>, Ts>
    && ...)>
    conjunction<is_nothrow_constructible<decay_t<Ts>, Ts>...>;

template <class... Ts>
using conjunction = boolean_constant<(Ts::value && ...)>;

using potentially-throwing-completions =
    conditional_t<
        error_types_of_t<copy_cvref_t<S2, S>, E, all-nothrow-decay-
            copyable>::value &&
            value_types_of_t<copy_cvref_t<S2, S>, E, all-nothrow-
                decay-copyable, conjunctionconjunction>::value,
        completion_signatures<>,
        completion_signatures<set_error_t(exception_ptr)>;

template <class...>
using no-completions = completion_signatures<>;

template <class... Ts>
using value-completions =
    completion_signatures<set_value_t(decay_t<Ts>&&...)>;

template <class T>
using error-completions =
    completion_signatures<set_error_t(decay_t<T>&&)>;
```

- For a sender *t* returned from *schedule\_from(sch, s)*, *get\_env(t)* shall return a queryable object *q* such that *get\_domain(q)* is expression-equivalent to *get\_domain(sch)* and *get\_completion\_scheduler<CPO>(q)* returns a copy of *sch*, where *CPO* is either *set\_value\_t* or *set\_stopped\_t*. The *get\_completion\_scheduler<set\_error\_t>* query is not implemented, as the scheduler cannot be guaranteed in case an error is thrown while trying to schedule work on the given scheduler object. For all other query objects *Q* whose type satisfies *forwarding-query*, the expression *Q(q, args...)* shall be equivalent to *Q(get\_env(s), args...)*.



[ Editor's note: Update §11.9.6.6 [exec.then] (with analogous changes to §11.9.6.7 [exec.upon.error] and §11.9.6.8 [exec.upon.stopped]) as follows: ]

1. then attaches an invocable as a continuation for an input sender's value completion operation.
2. The name then denotes a customization point object. For some subexpressions *s* and *f*, let *S* be `decltype((s))`, let *F* be the decayed type of *f*, and let ~~*f*~~*f*2 be an xvalue referring to an object decay-copied from *f*. If *S* does not satisfy sender, or *F* does not model *movable-value*, then `then(s, f)` is ill-formed. Otherwise, the expression `then(s, f)` is expression-equivalent to:

```
transform_sender(  
    get-sender-domain(s),  
    make-then-sender(f, s));
```

where *make-then-sender*(*f*, *s*) is expression-equivalent to *make-sender*(*then*, *f*, *s*) and returns a sender object *s*2 that behaves as follows:

1. `tag_invoke(then, get_completion_scheduler<set_value_t>(get_env(s)), s, f)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

2. Otherwise, `tag_invoke(then, s, f)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

1. ~~Otherwise, constructs a sender *s*2:~~ When *s*2 is connected with some receiver *out\_r*, it:

1. Constructs a receiver *r* such that:

1. When `set_value(r, args...)` is called, let *v* be the expression `invoke(ff2, args...)`. If `decltype(v)` is void, calls `set_value(out_r)`; otherwise, it calls `set_value(out_r, v)`. If any of these throw an exception, it catches it and calls `set_error(out_r, current_exception())`. If any of these expressions would be ill-formed, the expression `set_value(r, args...)` is ill-formed.

2. `set_error(r, e)` is expression-equivalent to `set_error(out_r, e)`.

3. `set_stopped(r)` is expression-equivalent to `set_stopped(out_r)`.

2. Returns an expression-equivalent to `connect(s, r)`.

3. ~~Let *compl\_sig\_t*<*Tag*, *Args...*> name the type *Tag*(*...*) if *Args...* is a template paramter pack containing the single type void; otherwise, *Tag*(*Args...*).~~ Given subexpressions *s*2 and *e* where *s*2 is a sender returned from *then* or a copy of such, let *S*2 be `decltype((s2))` and let *E* be `decltype((e))`. The type of `tag_invoke(get_completion_signatures, s2, e)` shall be **equivalent to**:

```
make_completion_signatures<>  
    copy_cvref_t<S2, S>, E, set-error-signature,  
    set-value-completions> ÷
```

where *set-value-completions* is ~~an alias for~~ the alias template:

```
template<class... As>
  set-value-completions =
    completion_signatures<compl_sig_t<set_value_t, SET-
      VALUE-SIG(invoke_result t<F, As...>)>>
```

and `set-error-signature` is an alias for `completion_signatures<set_error_t(exception_ptr)>` if any of the types in the `type-list` named by `value_types_of_t<copy_cvref_t<S2, S>, E, potentially-throwing, type-list>` are `true_type`; otherwise, `completion_signatures<>`, where `potentially-throwing` is the ~~template~~ alias template:

```
template<class... As>
  using potentially-throwing =
    bool_constant<!is_nothrow_invocable_v<F, As...>>
    negation<is_nothrow_invocable<F, As...>>;
```

2. ~~If the function selected above~~ Let  $S$  be the result of calling `then(s, f)` or a copy of such. If  $S$  is connected with a receiver  $R$  with environment  $E$  such that `transform_sender(get-env-domain(E), S, E)` does not return a sender that: [ Editor's note: reformed as a list for comprehensibility: ]

- invokes  $f$  with the value result datums of  $s$ ,
  - ~~using~~ uses  $f$ 's return value as ~~the sender's~~  $S$ 's value completion, and
  - forwards the non-value completion operations to  $R$  unchanged,
- then the behavior of calling ~~`then(s, f)`~~ `connect(S, R)` is undefined.

[ Editor's note: Change §11.9.6.9 [exec.let] as follows: ]

1. `let_value` transforms a sender's value completion into a new child asynchronous operation.  
`let_error` transforms a sender's error completion into a new child asynchronous operation.  
`let_stopped` transforms a sender's stopped completion into a new child asynchronous operation.
2. [ Editor's note: Copied from below: ] Let the expression `let-cpo` be one of `let_value`, `let_error`, or `let_stopped` and let `set-cpo` be the completion function that corresponds to `let-cpo` (set value for `let_value`, etc.). For subexpressions  $se$  and  $re$ , let `inner-env( $se$ ,  $re$ )` be an environment  $e$  such that:

— `get_domain( $e$ )` is expression-equivalent to the first well-formed expression below:

- `get_domain( $se$ )`
- `get_domain(get_completion_scheduler<set-cpo-t>( $se$ ))`, where `set-cpo-t` is the type of `set-cpo`.
- `get-env-domain( $re$ )`

— `get_scheduler( $e$ )` is expression-equivalent to the first well-formed expression below:

- `get_completion_scheduler<set-cpo-t>( $se$ )`

— `get_scheduler(re)`

or if neither of them are, `get_scheduler(e)` is ill-formed.

— For all other query objects  $Q$  and arguments  $\text{args} \dots$ ,  $Q(E, \text{args} \dots)$  is expression-equivalent to  $Q(\text{RE}, \text{args} \dots)$ .

3. The names `let_value`, `let_error`, and `let_stopped` denote customization point objects. ~~Let the expression `let-cpo` be one of `let_value`, `let_error`, or `let_stopped`.~~ For subexpressions  $s$  and  $f$ , let  $S$  be `decltype((s))`, let  $F$  be the decayed type of  $f$ , and let  $f'$  be an xvalue that refers to an object decay-copied from  $f$ . If  $S$  does not satisfy `sender`, the expression `let-cpo(s, f)` is ill-formed. If  $F$  does not satisfy `invocable`, the expression `let_stopped(s, f)` is ill-formed. Otherwise, the expression `let-cpo(s, f)` is expression-equivalent to:

```
transform_sender(  
    get_sender_domain(s),  
    make-let-sender(f, s));
```

where `make-let-sender(f, s)` is expression-equivalent to `make-sender(let-cpo, f, s)` and returns a sender object  $s_2$  that behaves as follows:

1. When  $s_2$  is connected to some receiver  $\text{out}_r$ , it:

1. `tag_invoke(let-cpo, get_completion_scheduler<set_value_t>(get_env(s)), s, f)`, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

2. Otherwise, `tag_invoke(let-cpo, s, f)`, if that expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

3. Otherwise, given a receiver  $\text{out}_r$  and an lvalue  $\text{out}_r'$  referring to an object decay-copied from  $\text{out}_r$ .

1. For `let_value`, let `set-cpo` be `set_value`. For `let_error`, let `set-cpo` be `set_error`. For `let_stopped`, let `set-cpo` be `set_stopped`. Let *completion-function* be one of `set_value`, `set_error`, or `set_stopped`.

1. Decay-copies  $\text{out}_r$  into  $\text{op\_state2}$  (see below).  $\text{out}_r$  is an xvalue referring to the copy of  $\text{out}_r$ .

2. ~~Let  $r$  be an rvalue of a receiver type  $R$ .~~ Constructs a receiver  $r$  such that such that:

1. When `set-cpo(r, args...)` is called, the receiver  $r$  decay-copies  $\text{args} \dots$  into  $\text{op\_state2}$  as  $\text{args}' \dots$ , then calls `invoke(f', args'...)` resulting in a sender  $s_3$ . It then calls `connect(s_3, std::move(out_r')out_r3)`, resulting in an operation state  $\text{op\_state3}$ , where  $\text{out}_r$  is a receiver described below.  $\text{op\_state3}$  is saved as a part of  $\text{op\_state2}$ . It then calls `start(op_state3)`. If any of these throws an exception, it catches it and calls `set_error(std::move(out_r')out_r2, current_exception())`. If any of these expressions would be ill-formed, `set-cpo(r, args...)` is ill-formed.

2. ~~completion-function~~CF(r, args...) is expression-equivalent to ~~completion-function(std::move(out\_r'), args...)~~ when completion-function is different from set-cpo CF(out\_r2, args...), where CF is a completion function other than set-cpo.
  3. get\_env(r) is expression-equivalent to get\_env(out\_r).
  4. out\_r3 is a receiver that forwards its completion operations to out\_r2 and for which get\_env(out\_r3) returns inner-env(get\_env(s), get\_env(out\_r2)).
  3. ~~let-cpo(s, f) returns a sender s2 such that:~~Calls connect(s, r) resulting in an operation state op\_state2. [ Editor's note: The formatting is changed here. ] If the expression connect(s, r) is ill-formed, connect(s2, out\_r) is ill-formed.
  4. ~~Otherwise, let op\_state2 be the result of connect(s, r). connect(s2, out\_r) returns~~Returns an operation state op\_state that stores op\_state2. start(op\_state) is expression-equivalent to start(op\_state2).
4. Given subexpressions s2 and e, where s2 is a sender returned from *let-cpo*(s, f) or a copy of such, let S2 be decltype((s2)), let E be decltype((e)), and let DS be copy\_cvref\_t<S2, S>. Then the type of tag\_invoke(get\_completion\_signatures, s2, e) is specified as follows:
1. If sender\_in<DS, E> is false, the expression tag\_invoke(get\_completion\_signatures, s2, e) is ill-formed.
  2. Otherwise, let Sigs... be the set of template arguments of the completion\_signatures specialization named by completion\_signatures\_of\_t<DS, E>, let Sigs2... be the set of function types in Sigs... whose return type is *set-cpo*, and let Rest... be the set of function types in Sigs... but not Sigs2....
  3. For each Sig2<sub>i</sub> in Sigs2..., let Vs<sub>i</sub>... be the set of function arguments in Sig2<sub>i</sub> and let S3<sub>i</sub> be invoke\_result\_t<F, decay\_t<Vs<sub>i</sub>>...>. If S3<sub>i</sub> is ill-formed, or if get-sender-domain(declval<S3<sub>i</sub>>()) has a different type than get-sender-domain(s), or if sender\_in<S3<sub>i</sub>, ~~EE2~~> is not satisfied where E2 is the type of inner-env(get\_env(s), e), then the expression tag\_invoke(get\_completion\_signatures, s2, e) is ill-formed.
  4. Otherwise, let Sigs3<sub>i</sub>... be the set of template arguments of the completion\_signatures specialization named by completion\_signatures\_of\_t<S3<sub>i</sub>, ~~EE2~~>. Then the type of tag\_invoke(get\_completion\_signatures, s2, e) shall be equivalent to completion\_signatures<Sigs3<sub>0</sub>..., Sigs3<sub>1</sub>..., ... Sigs3<sub>n-1</sub>..., Rest..., set\_error\_t(exception\_ptr)>, where n is sizeof...(Sigs2).
5. Let s and e be subexpressions such that S is decltype((s)) and E is decltype((e)). If sender-for<S, let-cpo-t> is false where let-cpo-t is the type of let-cpo, then the expression *let-cpo-t().transform\_env*(s, e) is ill-formed. Otherwise, it is equal to *inner-env*(get\_env(s), e).

6. If a sender  $S$  returned from  $let-cpo(s, f)$  is connected to a receiver  $R$  with environment  $E$  such that  $transform\_sender(get-env-domain(E), S, E)$  does not return a sender that [ Editor's note: reformatted as a list for comprehensibility ]:

- invokes  $f$  when  $set-cpo$  is called with  $s$ 's result datums, ~~and~~
  - makes its completion dependent on the completion of a sender returned by  $f$ , and
  - propagates the other completion operations sent by  $s$ ,
- the behavior of calling  ~~$let-cpo(s, f)$~~  $connect(S, R)$  is undefined.

[ Editor's note: Change §11.9.6.10 [exec.bulk] as follows: ]

1. `bulk` runs a task repeatedly for every index in an index space.
2. The name `bulk` denotes a customization point object. For some subexpressions  $s$ ,  $shape$ , and  $f$ , let  $S$  be `decltype((s))`,  $Shape$  be `decltype((shape))`, and  $F$  be `decltype((f))`. If  $S$  does not satisfy `sender` or  $Shape$  does not satisfy `integral`, `bulk` is ill-formed. Otherwise, the expression `bulk(s, shape, f)` is expression-equivalent to:

```
transform_sender(
  get_sender_domain(s),
  make_bulk_sender(tuple-like{shape, f}, s));
```

where `make_bulk_sender(t, s)` is expression-equivalent to `make_sender(bulk, t, s)` for a subexpression  $t$  and returns a sender object  $s2$  that behaves as follows:

1. `tag_invoke(bulk, get_completion_scheduler<set_value_t>(get_env(s)), s, shape, f)`, if that expression is valid.
  - *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.
2. Otherwise, `tag_invoke(bulk, s, shape, f)`, if that expression is valid.
  - *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.
1. ~~Otherwise, constructs a sender  $s2$ :~~ When  $s2$  is connected with some receiver `out_r`, it:

1. Constructs a receiver  $r$ :

1. When `set_value(r, args...)` is called, calls `f(i, args...)` for each  $i$  of type `Shape` from `0` to `shape`, then calls `set_value(out_r, args...)`. If any of these throws an exception, it catches it and calls `set_error(out_r, current_exception())`. If any of these expressions are ill-formed, `set_value(r, args...)` is ill-formed.
2. When `set_error(r, e)` is called, calls `set_error(out_r, e)`.
3. When `set_stopped(r)` is called, calls `set_stopped(out_r, e)`.

2. Calls `connect(s, r)`, which results in an operation state `op_state2`.

3. Returns an operation state `op_state` that contains `op_state2`. When `start(op_state)` is called, calls `start(op_state2)`.

4. Given subexpressions *s2* and *e* where *s2* is a sender returned from *bulk* or a copy of such, let *S2* be `decltype((s2))`, let *E* be `decltype((e))`, let *DS* be `copy_cvref_t<S2, S>`, let *Shape* be `decltype((shape))` and let *nothrow-callable* be the alias template:

```
template<class... As>
using nothrow-callable =
    bool_constant<is_nothrow_invocable_v<decay_t<F>&,
        Shape, As...>>;
```

1. If any of the types in the *type-list* named by `value_types_of_t<DS, E, nothrow-callable, type-list>` are `false_type`, then the type of `tag_invoke(get_completion_signatures, s2, e)` shall be **equivalent to**:

```
make_completion_signatures<
    DS, E, completion_signatures<set_error_t(exception_ptr)>>
```

2. Otherwise, the type of `tag_invoke(get_completion_signatures, s2, e)` shall be **equivalent to** `completion_signatures_of_t<DS, E>`.

2. ~~If the function selected above~~ Let *S* be the result of calling `bulk(s, shape, f)` or a copy of such. If *S* is connected to a receiver *R* with environment *E* such that `transform_sender(get-env-domain(E), S, E)` does not return a sender that invokes `f(i, args...)` for each *i* of type *Shape* from 0 to *shape* where *args* is a pack of subexpressions referring to the value completion result datums of the input sender, or does not execute a value completion operation with said datums, the behavior of calling `bulk(s, shape, f)connect(S, R)` is undefined.

[ Editor's note: Change §11.9.6.11 [exec.split] as follows: ]

1. *split* adapts an arbitrary sender into a sender that can be connected multiple times.
2. Let *split-env* be the type of an environment such that, given an instance *e*, the expression `get_stop_token(e)` is well-formed and has type `stop_token`.
3. The name *split* denotes a customization point object. For some subexpression *s*, let *S* be `decltype((s))`. If `sender_in<S, split-env>` or `constructible_from<decay_t<env_of_t<S>>>, env_of_t<S>>` is `false`, *split* is ill-formed. Otherwise, the expression `split(s)` is expression-equivalent to:

```
transform_sender(
    get-sender-domain(s),
    make-sender(split, s));
```

1. `tag_invoke(split, get_completion_scheduler<set_value_t>(get_env(s)), s)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

2. Otherwise, `tag_invoke(split, s)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

1. Let *s* be a subexpression such that *S* is `decltype((s))`, and let *e...* be a pack of subexpressions such that `sizeof...(e) <= 1` is true. If `sender-for<S, split t>` is

false, then the expression `split t().transform sender(s, e...)` is ill-formed; otherwise, it returns ~~Otherwise, constructs~~ a sender `s2` ~~, which that~~:

1. Creates an object `sh_state` that [ Editor's note: ... as before ]

[Change §11.9.6.12 [exec.when.all] as follows:]

1. `when_all` and `when_all_with_variant` both adapt multiple input senders into a sender that completes when all input senders have completed. `when_all` only accepts senders with a single value completion signature and on success concatenates all the input senders' value result datums into its own value completion operation. `when_all_with_variant(s...)` is semantically equivalent to `when_all(into_variant(s)...) , where s is a pack of subexpressions of sender types.`
2. The names `when_all` and `when_all_with_variant` ~~denotes a~~ customization point objects. For some subexpressions `si...`, let `Si...` be `decltype((si))...`. The expressions `when_all(si...)` ~~is~~ and `when_all_with_variant(si...)` are ill-formed if any of the following is true:

- If the number of subexpressions `si...` is 0, or
- If any type `Si` does not satisfy sender.
- If the expression `get_sender_domain(si)` has a different type for any other value of `i`.

Otherwise, those expressions have the semantics specified below.

[ Editor's note: The following paragraph becomes numbered and subsequent paragraphs are renumbered. ]

3. ~~Otherwise, the~~ The expression `when_all(si...)` is expression-equivalent to:

```
transform_sender(  
    get_sender_domain(s0),  
    make_when_all_sender(s0, ... sn-1))
```

where `make_when_all_sender(si...)` is expression-equivalent to `make_sender(when_all, unspecified, si...)` and returns a sender object `w` of type `w` that behaves as follows:

1. `tag_invoke(when_all, si...)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that sends a concatenation of values sent by `si...` when they all complete with `set_value`, the behavior of calling `when_all(si...)` is undefined.
  - *Mandates:* The type of the `tag_invoke` expression above satisfies sender.
2. ~~Otherwise, constructs a sender `w` of type `w`.~~ When `w` is connected with some receiver `out_r` of type `OutR`, it returns an operation state `op_state` specified as below:

1. For each sender `si`, ... [ Editor's note: ... as before ]

...

4. ~~The name `when_all_with_variant` denotes a customization point object. For some subexpressions  $s_i$ , let  $S$  be `decltype((s))`. If any type  $S_i$  in  $S$  does not satisfy sender, `when_all_with_variant` is ill-formed. Otherwise, the~~ The expression `when_all_with_variant( $s_i$ ...)` is expression-equivalent to:

```
transform_sender(
    get_sender_domain(s0),
    make_sender(when_all_with_variant, unspecified, s0, ... sn-1))
```

1. `tag_invoke(when_all_with_variant, s...)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that, when connected with a receiver of type  $R$ , sends the types `into_variant_type< $S$ , env_of_t< $R$ >>... when they all complete with set_value, the behavior of calling when_all( $s_i$ ...) is undefined.`

— *Mandates:* The type of the `tag_invoke` expression above satisfies sender.

2. Otherwise, `when_all(into_variant(s)...)...`.

5. Let  $s$  and  $e$  be subexpressions such that  $S$  is `decltype((s))`. If `sender_for< $S$ , when_all_with_variant_t>` is false, then the expression `when_all_with_variant_t().transform_sender(s, e)` is ill-formed; otherwise, it is equal to:

```
const auto& env = e;
auto domain = get_env_domain(env);
auto [tag, data, ...child] = s;
auto tfx = make_transformer_fn(domain, env);
return tfx(when_all)(tfx(into_variant)(std::move(child))...);
```

[ *Note:* This causes the `when_all_with_variant(s...)` sender to become `when_all(into_variant(s)...)...` when it is connected with a receiver whose execution domain does not customize `when_all_with_variant`. — *end note* ]

5. ~~For a sender  $s_2$  returned from `when_all` or `when_all_with_variant`, `get_env(s2)` shall return an instance of a class equivalent to `empty_env`. Given a pack of subexpressions  $s_i$ , let  $S$  be an object returned from `when_all(s...)` or `when_all_with_variant(s...)` or a copy of such, and let  $E$  be the environment object returned from `get_env( $S$ )`. Given a query object  $Q$ , `tag_invoke(Q, E)` is expression-equivalent to `get_sender_domain( $s_0$ )` when  $Q$  is `get_domain`; otherwise, it is ill-formed.~~

[ Editor's note: Change §11.9.6.13 [exec.transfer.when.all] as follows: ]

1. `transfer_when_all` and `transfer_when_all_with_variant` both adapt multiple input senders into a sender that completes when all input senders have completed, ensuring the input senders complete on the specified scheduler. `transfer_when_all` only accepts senders with a single value completion signature and on success concatenates all the input senders' value result datums into its own value completion operation; `transfer_when_all(scheduler, input-senders...)` is semantically equivalent to `transfer(when_all(input-senders...), scheduler)`. `transfer_when_all_with_variant(scheduler, input-senders...)` is semantically equivalent to `transfer_when_all(scheduler, into_variant(input-senders)...)...`. [ *Note:* These customizable composite algorithms can allow for more efficient customizations in some cases. — *end note* ]



2. The name `transfer_when_all` denotes a customization point object. For some subexpressions `sch` and `s...`, let `Sch` be `decltype(sch)` and `S` be `decltype(s)`. If `Sch` does not satisfy `scheduler`, or any type `Si` in `S...` does not satisfy `sender`, `transfer_when_all` is ill-formed. Otherwise, the expression `transfer_when_all(sch, s...)` is expression-equivalent to:

```
return transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(transfer_when_all, sch, s...));
```

1. `tag_invoke(transfer_when_all, sch, s...)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that sends a concatenation of values sent by `s...` when they all complete with `set_value`, or does not send its completion operation, other than ones resulting from a scheduling error, on an execution agent belonging to the associated execution resource of `sch`, the behavior of calling `transfer_when_all(sch, s...)` is undefined.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

2. Otherwise, `transfer_when_all(s..., sch)`.

3. Let `s` and `e` be subexpressions such that `S` is `decltype(s)`. If `sender_for<S, transfer_when_all_t>` is false, then the expression `transfer_when_all_t().transform_sender(s, e)` is ill-formed; otherwise, it is equal to:

```
const auto& env = e;
auto domain = get_env_domain(env);
auto [tag, data, ...child] = s;
auto tfx = make_transformer_fn(domain, env);
return tfx(transfer)(tfx(when_all)(std::move(child)...),
    std::move(data));
```

[ *Note:* This causes the `transfer_when_all(sch, s...)` sender to become `transfer_when_all(s..., sch)` when it is connected with a receiver whose execution domain does not customize `transfer_when_all`. — *end note* ]

3. The name `transfer_when_all_with_variant` denotes a customization point object. For some subexpressions `sch` and `s...`, let `Sch` be `decltype(sch)` and let `S` be `decltype(s)`. If any type `Si` in `S...` does not satisfy `sender`, `transfer_when_all_with_variant` is ill-formed. Otherwise, the expression `transfer_when_all_with_variant(sch, s...)` is expression-equivalent to:

```
return transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(transfer_when_all_with_variant, sch, s...));
```

1. `tag_invoke(transfer_when_all_with_variant, s...)`, if that expression is valid. If the function selected by `tag_invoke` does not return a sender that, when connected with a receiver of type `R`, sends the types `into_variant_type<S, env_of_t<R>>...` when they all complete with `set_value`, the behavior of calling `transfer_when_all_with_variant(sch, s...)` is undefined.

— *Mandates:* The type of the `tag_invoke` expression above satisfies `sender`.

2. Otherwise, `transfer_when_all(sch, into_variant(s)...)...`.

4. Let `s` and `e` be subexpressions such that `S` is `decltype((s))`. If `sender-for<S, transfer_when_all_with_variantt>` is false, then the expression `transfer_when_all_with_variant_t().transform_sender(s, e)` is ill-formed; otherwise, it is equal to:

```
const auto& env = e;
auto domain = get-env-domain(env);
auto [tag, data, ...child] = s;
auto tfx = make-transformer-fn(domain, env);
return tfx(transfer_when_all)(std::move(data), tfx(into_variant)
    (std::move(child)...));
```

[ *Note:* This causes the `transfer_when_all_with_variant(sch, s...)` sender to become `transfer_when_all(sch, into_variant(s)...` when it is connected with a receiver whose execution domain does not customize `transfer_when_all_with_variant`. — *end note* ]

5. For a sender `t` returned from `transfer_when_all(sch, s...)` or `transfer_when_all_with_variant(sch, s...)`, `get_env(t)` shall return a queryable object `q` such that `get_domain(q)` shall be expression-equivalent to `get_domain(sch)`, and `get_completion_scheduler<CPO>(q)` returns a copy of `sch`, where `CPO` is either `set_value_t` or `set_stopped_t`. The `get_completion_scheduler<set_error_t>` query is not implemented, as the scheduler cannot be guaranteed in case an error is thrown while trying to schedule work on the given scheduler object.

[ Editor's note: Change §11.9.6.14 [exec.into.variant] as follows: ]

1. `into_variant` adapts a sender with multiple value completion signatures into a sender with just one consisting of a variant of tuples.
2. The template `into-variant-type` computes the type sent by a sender returned from `into_variant`.

```
template<class S, class E>
    requires sender_in<S, E>
    using into-variant-type =
        value_types_of_t<S, E>;
```

3. `into_variant` is a customization point object. For some subexpression `s`, let `S` be `decltype((s))`. If `S` does not satisfy `sender`, `into_variant(s)` is ill-formed. Otherwise, `into_variant(s)` is expression-equivalent to:

```
transform_sender(
    get-sender-domain(s),
    make-into-variant-sender(s))
```

where `make-into-variant-sender(s)` is expression-equivalent to `make-sender(into_variant, unspecified, s)` and returns a sender object `s2`: that behaves as follows:

[ Editor's note: Reformatting here ]

1. When `s2` is connected with some receiver `out_r`, it:
2. Constructs a receiver `r`:

1. If `set_value(r, ts...)` is called, calls `set_value(out_r, into-variant-type<S, env_of_t<decltype((r))>>(decayed-tuple<decltype(ts)...>(ts...)))`. If this expression throws an exception, calls `set_error(out_r, current_exception())`.
2. `set_error(r, e)` is expression-equivalent to `set_error(out_r, e)`.
3. `set_stopped(r)` is expression-equivalent to `set_stopped(out_r)`.
3. Calls `connect(s, r)`, resulting in an operation state `op_state2`.
4. Returns an operation state `op_state` that contains `op_state2`. When `start(op_state)` is called, calls `start(op_state2)`.
5. Given subexpressions `s2` and `e` [...] [ Editor's note: ...as before ]

[ Editor's note: Change §11.9.6.15 [exec.stopped.as.optional] as follows: ]

1. `stopped_as_optional` maps an input sender's stopped completion operation into the value completion operation as an empty optional. The input sender's value completion operation is also converted into an optional. The result is a sender that never completes with stopped, reporting cancellation by completing with an empty optional.
2. The name `stopped_as_optional` denotes a customization point object. For some subexpression `s`, let `S` be `decltype((s))`. ~~Let `get_env_sender` be an expression such that, when it is connected with a receiver `r`, start on the resulting operation state completes immediately by calling `set_value(r, get_env(r))`.~~ The expression `stopped_as_optional(s)` is expression-equivalent to:

```
transform_sender(
    get-sender-domain(s),
    make-sender(stopped_as_optional, unspecified, s))

let_value(
    get-env-sender,
    [<class E>(const E&) requires single-sender<S, E> {
        return let_stopped(
            then(s,
                [<class T>(T&& t) {
                    return optional<decay_t<single-sender-value-type<S, E>>>{
                        std::forward<T>(t)
                    };
                }
            ),
            [&] () noexcept {
                return just(optional<decay_t<single-sender-value-type<S,
                    E>>>{});
            }
        );
    }
);
```

3. Let `s` and `e` be subexpressions such that `S` is `decltype((s))` and `E` is `decltype((e))`. If either `sender-for<S, stopped_as_optional_t>` or `single-sender<S, E>` is false, then the expression `stopped_as_optional_t().transform_sender(s, e)` is ill-formed; otherwise, it is equal to:

```

const auto& env = e;
auto domain = get-env-domain(env);
auto [tag, data, child] = s;
using V = single-sender-value-type<S, E>;
auto tfx = make-transformer-fn(domain, env);
return tfx(let_stopped)(
    tfx(then)(std::move(child),
        [<class T>(T&& t) { return optional<V>
            (std::forward<T>(t)); })),
    []() noexcept { return just(optional<V>()); });

```

[ Editor's note: Change §11.9.6.16 [*exec.stopped.as.error*] as follows: ]

1. *stopped\_as\_error* maps an input sender's stopped completion operation into an error completion operation as a custom error type. The result is a sender that never completes with stopped, reporting cancellation by completing with an error.
2. The name *stopped\_as\_error* denotes a customization point object. For some subexpressions *s* and *e*, let *S* be *decltype*((*s*)) and let *E* be *decltype*((*e*)). If the type *S* does not satisfy sender or if the type *E* doesn't satisfy *movable-value*, *stopped\_as\_error*(*s*, *e*) is ill-formed. Otherwise, the expression *stopped\_as\_error*(*s*, *e*) is expression-equivalent to:

```
let_stopped(s, [] { return just_error(e); })
```

```

transform_sender(
    get-sender-domain(s),
    make-sender(stopped_as_error, e, s))

```

3. Let *s* and *e* be subexpressions such that *S* is *decltype*((*s*)) and *E* is *decltype*((*e*)). If *sender-for*<*S*, *stopped\_as\_error\_t*> is false, then the expression *stopped\_as\_error\_t*().*transform\_sender*(*s*, *e*) is ill-formed; otherwise, it is equal to:

```

const auto& env = e;
auto domain = get-env-domain(env);
auto [tag, data, child] = s;
auto tfx = make-transformer-fn(domain, env);
return tfx(let_stopped)(
    std::move(child),
    [err = std::move(data)]() mutable { return
        just_error(std::move(err)); });

```

[ Editor's note: Change §11.9.6.17 [*exec.ensure.started*] as follows: ]

1. *ensure\_started* eagerly starts the execution of a sender, returning a sender that is usable as input to additional sender algorithms.
2. Let *ensure-started-env* be the type of an execution environment such that, given an instance *e*, the expression *get\_stop\_token*(*e*) is well-formed and has type *stop\_token*.
3. The name *ensure\_started* denotes a customization point object. For some subexpression *s*, let *S* be *decltype*((*s*)). If *sender\_in*<*S*, *ensure-started-env*> or *constructible\_from*<*decay\_t*<*env\_of\_t*<*S*>>, *env\_of\_t*<*S*>> is false, *ensure\_started*(*s*) is ill-formed. Otherwise, the expression *ensure\_started*(*s*) is expression-equivalent to:

```

transform_sender(
    get-sender-domain(s),
    make-sender(ensure_started, s));

```

1. `tag_invoke(ensure_started, get_completion_scheduler<set_value_t>(get_env(s)), s)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

2. Otherwise, `tag_invoke(ensure_started, s)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above satisfies sender.

1. Let  $s$  be a subexpression such that  $S$  is `decltype((s))`, and let  $e...$  be a pack of subexpressions such that `sizeof...(e) <= 1` is true. If `sender-for<S, ensure_started t>` is false, then the expression `ensure_started t().transform_sender(s, e...)` is ill-formed; otherwise, it returns ~~Otherwise, constructs~~ a sender  $s_2$ , ~~which~~ that:

1. Creates an object `sh_state` that [ Editor's note: ... as before ]

[ Editor's note: Change §11.9.7.1 [exec.start.detached] as follows: ]

1. `start_detached` eagerly starts a sender without the caller needing to manage the lifetimes of any objects.

2. The name `start_detached` denotes a customization point object. For some subexpression  $s$ , let  $S$  be `decltype((s))`. If  $S$  does not satisfy ~~sender~~ `sender_in<empty_env>`, `start_detached` is ill-formed. Otherwise, the expression `start_detached(s)` is expression-equivalent to:

```
apply_sender(get_sender_domain(s), start_detached, s)
```

— *Mandates*: The type of the expression above is `void`.

If the ~~function-selected~~ expression above does not eagerly start the sender  $s$  after connecting it with a receiver that ignores value and stopped completion operations and calls `terminate()` on error completions, the behavior of calling `start_detached(s)` is undefined.

1. `tag_invoke(start_detached, get_completion_scheduler<set_value_t>(get_env(s)), s)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above is `void`.

2. Otherwise, `tag_invoke(start_detached, s)`, if that expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above is `void`.

3. Otherwise, let  $R$  be the type of a receiver, let  $r$  be an rvalue of type  $R$ , and let  $cr$  be a lvalue reference to `const R` such that:

— The expression `set_value(r)` is not potentially-throwing and has no effect,

— For any subexpression  $e$ , the expression `set_error(r, e)` is expression-equivalent to `terminate()`,

— The expression `set_stopped(r)` is not potentially-throwing and has no effect, and

— The expression `get_env(cr)` is expression-equivalent to `empty_env{}`.

Calls `connect(s, r)`, resulting in an operation state `op_state`, then calls `start(op_state)`.

3. Let `s` be a subexpression such that `S` is `decltype((s))`, and let *detached-receiver* and *detached-operation* be the following exposition-only class types:

```
struct detached-receiver {
    using is_receiver = unspecified;
    detached-operation* op; // exposition only

    friend void tag_invoke(set_value_t, detached-receiver&& self)
        noexcept { delete self.op; }
    friend void tag_invoke(set_error_t, detached-receiver&&, auto&&)
        noexcept { terminate(); }
    friend void tag_invoke(set_stopped_t, detached-receiver&& self)
        noexcept { delete self.op; }
    friend empty_env tag_invoke(get_env_t, const detached-receiver&)
        noexcept { return {}; }
};

struct detached-operation {
    connect_result_t<S, detached-receiver> op; // exposition only

    explicit detached-operation(S&& s)
        : op(connect(std::forward<S>(s), detached-receiver{this}))
    {}
};
```

If `sender_to<S, detached-receiver>` is false, then the expression `start_detached_t().apply_sender(s)` is ill-formed; otherwise, it is expression-equivalent to:

```
start((new detached-operation(s))->op)
```

[ Editor's note: Change §11.9.7.2 [exec.sync.wait] as follows: ]

1. [...]

4. The name `this_thread::sync_wait` denotes a customization point object. For some subexpression `s`, let `S` be `decltype((s))`. If `sender_in<S, sync-wait-env>` is false, or ~~the number of the arguments completion signatures of `t<S, sync-wait-env>::value_types` passed into the variant template parameter is not 1~~ if the type completion signatures of `t<S, sync-wait-env, type-list, type identity t>` is ill-formed, `this_thread::sync_wait(s)` is ill-formed. Otherwise, `this_thread::sync_wait(s)` is expression-equivalent to:

```
apply_sender(get_sender_domain(s), sync_wait, s)
```

— *Mandates*: The type of expression above is `sync-wait-type<S, sync-wait-env>`.

1. `tag_invoke(this_thread::sync_wait, get_completion_scheduler<set_value_t>(get_env(s)), s)`, if this expression is valid.

— *Mandates*: The type of the `tag_invoke` expression above is `sync-wait-type<S, sync-wait-env>`.

2. Otherwise, `tag_invoke(this_thread::sync_wait, s)`, if this expression is valid and its type is.

— *Mandates:* The type of the `tag_invoke` expression above is *sync-wait-type*<*S*, *sync-wait-env*>.

5. Otherwise: Let *sync-wait-receiver* be a class type that satisfies receiver, let *r* be an xvalue of that type, and let *cr* be a const lvalue referring to *r* such that `get_env(cr)` has type *sync-wait-env*. If `sender_in<S, sync-wait-env>` is false, or if the type completion signatures of `t<S, sync-wait-env, type-list, type identity t>` is ill-formed, the expression `sync_wait t().apply_sender(s)` is ill-formed; otherwise, it has the following effects:

1. ~~Constructs a receiver *r*:~~

2. Calls `connect(s, r)`, resulting in an operation state `op_state`, then calls `start(op_state)`.

3. Blocks the current thread until a completion operation of *r* is executed. When it is:

1. If `set_value(r, ts...)` has been called, returns *sync-wait-type*<*S*, *sync-wait-env*>{*decayed-tuple*<*decltype*(*ts*)...>{*ts*...}}. If that expression exits exceptionally, the exception is propagated to the caller of `sync_wait`.

2. If `set_error(r, e)` has been called, let *E* be the decayed type of *e*. If *E* is `exception_ptr`, calls `std::rethrow_exception(e)`. Otherwise, if the *E* is `error_code`, throws `system_error(e)`. Otherwise, throws *e*.

3. If `set_stopped(r)` has been called, returns *sync-wait-type*<*S*, *sync-wait-env*>{ }.

6. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For some subexpression *s*, let *S* be the type of `into_variant(s)`. If `sender_in<S, sync-wait-env>` is false, `this_thread::sync_wait_with_variant(s)` is ill-formed. Otherwise, `this_thread::sync_wait_with_variant(s)` is expression-equivalent to:

```
apply_sender(get_sender_domain(s), sync_wait_with_variant, s)
```

— *Mandates:* The type of expression above is *sync-wait-with-variant-type*<*S*, *sync-wait-env*>.

1. `tag_invoke(this_thread::sync_wait_with_variant, get_completion_scheduler<set_value_t>(get_env(s)), s)`, if this expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above is *sync-wait-with-variant-type*<*S*, *sync-wait-env*>.

2. Otherwise, `tag_invoke(this_thread::sync_wait_with_variant, s)`, if this expression is valid.

— *Mandates:* The type of the `tag_invoke` expression above is *sync-wait-with-variant-type*<*S*, *sync-wait-env*>.

6. ~~Otherwise,~~ The expression `sync wait with variant t().apply_sender(s)` is expression-equivalent to `this_thread::sync_wait(into_variant(s))`.

[Update §11.10 [exec.execute] as follows:]

1. `execute` creates fire-and-forget tasks on a specified scheduler.
2. The name `execute` denotes a customization point object. For some subexpressions `sch` and `f`, let `Sch` be `decltype((sch))` and `F` be `decltype((f))`. If `Sch` does not satisfy `scheduler` or `F` does not satisfy `invocable`, `execute` is ill-formed. Otherwise, `execute` is expression-equivalent to:

```
apply_sender(  
    query-or-default(get_domain, sch, default_domain()),  
    execute, schedule(sch), f)
```

— *Mandates:* The type of the expression above is `void`.

1. `tag_invoke(execute, sch, f)`, if that expression is valid. If the function selected by `tag_invoke` does not invoke the function `f` (or an object decay-copied from `f`) on an execution agent belonging to the associated execution resource of `sch`, or if it does not call `std::terminate` if an error occurs after control is returned to the caller, the behavior of calling `execute` is undefined.

— *Mandates:* The type of the `tag_invoke` expression above is `void`.

3. ~~Otherwise,~~ For some subexpressions `s` and `f` where `F` is `decltype((f))`, if `F` does not satisfy `invocable`, the expression `execute t().apply_sender(s, f)` is ill-formed; otherwise, it is expression-equivalent to `start_detached(then(schedule(sch)s, f))`.

## § 6 References

[P1061R5] Barry Revzin, Jonathan Wakely. 2023-05-18. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r5>

[P2141R1] Antony Polukhin. 2023-05-03. Aggregates are named tuples.

<https://wg21.link/p2141r1>

[P2300R7] Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, and Bryce Adelstein Lelbach. `std::execution`.

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2300r7.html>

[stdexecgithub] stdexec.

<https://github.com/NVIDIA/stdexec>